

操作系统期中考试概念复习

基于2024年和2025年两份期中考试试题分析，按章节分类统计概念考察频率。

题型分布

题型	分值	说明
判断题	20分	每题2分，共10题
单项选择题	20分	每题2分，共10题
填空题	20分	每空2分
存储管理综合题	15分	页表计算+段页式地址转换
页表自映射	5分	虚拟地址计算
进程与线程简答题	10分	概念理解
进程同步/互斥应用题	10分	PV操作设计

一、操作系统概论

概念	考察频率	出现题目	考察形式
内核态与用户态	☆☆	2025判断1	判断题
操作系统主要功能	☆☆	2025判断2	判断题
微内核 vs 宏内核架构	☆☆	2025判断3	判断题
批处理系统	☆☆	2024选择1	选择题
多任务系统	☆☆	2024判断5	判断题
多用户系统	☆☆	2024判断5	判断题
MBR（主引导记录）	☆☆	2024判断6	判断题
引导扇区	☆☆	2024判断6	判断题
U-Boot引导程序	☆☆	2025填空4	填空题
系统调用	☆☆☆☆	2025选择4, 2024选择4	选择题
动态重定位	☆☆	2024判断9	判断题
ABI兼容性	☆☆	2024填空3	填空题
程序装载与链接	☆☆☆	2025填空6, 2024填空5	填空题
可执行文件内存分段	☆☆☆	2025填空6, 2024填空5	填空题

概念	考察频率	出现题目	考察形式
进程与程序关系	☆☆	2024判断7	判断题

高频考点详解

1. 内核态与用户态

- **核心目的：**保护操作系统核心资源，防止应用程序随意访问关键硬件
- **权限差异：**内核态可执行特权指令，用户态只能执行非特权指令

2. 微内核 vs 宏内核

- **宏内核 (Monolithic Kernel)：**所有OS功能都运行在内核态，体积大，性能高
- **微内核 (Microkernel)：**仅最基本功能在内核态，其他功能在用户态，**占用空间更小，更易扩展，但系统调用开销大**

3. 批处理系统特点

- **优点：**CPU利用率高，系统吞吐量大
- **缺点：**失去了交互性，用户无法与作业交互

4. 动态重定位

- 硬件在**每次内存访问时**动态实现地址转换
- 需要基址寄存器支持
- 优点：程序可在内存中移动，便于内存紧凑

二、内存管理

概念	考察频率	出现题目	考察形式
页表项 (PTE) 结构	★★★★	2025选择2, 综合题	选择+综合
多级页表	★★★★★	2025选择3, 2024选择3, 综合题	选择+计算
页面置换算法	★★★★★	2025选择8, 2024选择8, 填空1	选择+计算
FIFO算法	★★★★	2025填空1, 2024填空1	填空题
LRU算法	★★★	2024填空1	填空题
CLOCK算法	★★★	2025填空1	填空题
Belady现象	★★★	2024填空2	填空题
局部性原理	★★★	2025选择7	选择题
快表 (TLB)	★★★	2025选择9	选择题
反置页表	★★★	2024选择7	选择题

概念	考察频率	出现题目	考察形式
覆盖与交换	★★★★	2025选择10, 2024选择10	选择题
抖动/颠簸 (Thrashing)	★★★★	2025填空2, 2024判断4	判断+填空
工作集算法	★★★	2025选择8, 2024选择8	选择题
二次机会算法	★★★	2025选择8, 2024选择8	选择题
Aging算法	★★★	2025选择8, 2024选择8	选择题
WSClock算法	★★★	2025选择8, 2024选择8	选择题
段式内存管理	★★★★	2025判断4, 2024选择2, 综合题	各类题型
段页式内存管理	★★★★	2025综合题, 2024综合题	综合大题
缺页异常/中断	★★★	2025判断5	判断题
可变分区存储分配	★★★★	2025填空5, 2024填空8, 2024选择9	选择+填空
最佳适配 (Best Fit)	★★★	2024填空8	填空题
最坏适配 (Worst Fit)	★★★	2025填空5	填空题
内存碎片	★★★	2024填空4	填空题
页表自映射	★★★★	2025大题五, 2024大题五	计算大题
位图表示法	★★★	2025选择1	选择题
链表表示法	★★★	2025选择1	选择题
虚拟地址空间	★★★	2024选择2	选择题
物理地址空间	★★★	2024选择2	选择题
页面尺寸	★★★	2024选择2	选择题
页框号	★★★	2025选择2, 2024选择3	选择题

高频考点详解

1. 多级页表计算 (必考!)

- **32位系统 + 4KB页面:**
 - 页内偏移: 12位
 - 1级页表: 页表项数 = $2^{(32-12)} = 2^{20} = 1\text{M}$ 个
 - 无论分配多少内存, 1级页表都需要完整的1M个PTE
- **多级页表优势:** 减少页表占用内存 (只需分配实际用到的页表页)
- **多级页表劣势:** 级数越多, 平均访问内存时间越长 (需要多次查页表)

2. 页面置换算法

算法	特点	关键点
FIFO	简单，选择最早进入的页面淘汰	存在Belady现象
LRU	选择最久未使用的页面淘汰	性能好但实现开销大
CLOCK	给每个页面设置使用位，循环扫描	FIFO的改进，近似LRU
二次机会	检查使用位和修改位	CLOCK改进版
Aging	记录每个页面的"年龄"	LRU的高效软件近似
工作集	追踪进程最近访问的页面集合	驱逐不在工作集中的页面
WSClock	结合工作集和CLOCK思想	高效实用的算法

- **Belady现象**：增加页框数时，缺页中断次数反而增加的异常现象（仅FIFO存在）

3. 段页式地址转换流程

虚拟地址 → 虚拟段号 → 段表 → 页表基地址 → 虚拟页号 → 页表项 → 物理页框号 → 物理地址
 ↓
 页内偏移

4. 覆盖 vs 交换

特性	覆盖（Overlay）	交换（Swapping）
范围	同一程序/作业内部	不同作业/程序之间
实现者	应用程序员（编译器辅助）	操作系统
透明度	对程序员不透明	对程序员透明
目的	运行大于内存的程序	提高内存利用率和多道程序度

5. 抖动（Thrashing）

- **定义**：进程并发水平上升，常驻集减小，缺页率不断上升，频繁调页导致系统开销增大
- **特征**：CPU利用率很低，I/O设备繁忙
- **解决方法**：
 - 减少多道程序度（终止部分进程）
 - 增加物理内存
 - 采用工作集模型控制并发

6. 可变分区分配算法

- **最佳适配（Best Fit）**：选择可容纳作业的最小空闲区 → 产生大量外部碎片
- **最坏适配（Worst Fit）**：选择可容纳作业的最大空闲区 → 产生的碎片较大但数量少
- **首次适配（First Fit）**：从头开始找第一个满足的
- **下次适配（Next Fit）**：从上次位置继续找

7. 页表自映射计算技巧

- 一级页表起始虚拟地址 = 页表映射基址的高10位左移22位
- 自映射表项虚拟地址 = 一级页表基址 + 页表索引 × 4

三、进程与线程

概念	考察频率	出现题目	考察形式
进程与线程区别联系	★★★★★	2025大题六1, 2025选择5, 2024判断1,10	各类题型
进程上下文切换	★★★★★	2025大题六2, 2025选择4, 2024选择4, 2024判断8	各类题型
内核级线程	★★★	2025判断6	判断题
用户级线程	★★★	2024判断2	判断题
多对多线程模型	★★★	2025判断7	判断题
自旋锁（Spinlock）	★★★★	2025判断8, 2024大题七2	判断+编程
PV操作 / 信号量	★★★★★	2025判断9, 选择6, 大题七; 2024选择6, 大题七	全题型覆盖
临界区	★★★★	2025填空7, 2024填空6	填空+算法
Dekker算法	★★★	2025填空7	填空题
Peterson算法	★★★	2024填空6	填空题
进程三态模型	★★★★	2024大题六, 2025填空3	画图+填空
原子操作	★★★	2025选择6, 2024选择6	选择题
死锁	★★★	2024选择6	选择题
fork创建进程	★★★	2024判断3	判断题
线程栈独立性	★★★	2024判断1,10	判断题
test-and-set指令	★★★	2024大题七2	编程题
进程互斥	★★★★	2025大题七	算法设计
进程同步	★★★★	2025大题七	算法设计
读者写者问题	★★★★	2025大题七变种	算法设计

高频考点详解

1. 进程 vs 线程

特性	进程	线程
资源分配	资源分配基本单位	不拥有独立资源（共享进程资源）
调度执行	不是调度基本单位	CPU调度基本单位

特性	进程	线程
地址空间	独立地址空间	共享进程地址空间
上下文切换	开销大	开销小
栈	独立栈	各自拥有独立栈
寄存器上下文	独立	独立

- **联系**：一个进程可以拥有多个线程，一个线程只能属于一个进程
- **多线程优势**：更好释放多核性能，提高并发度
- **注意**：多线程**不总是**能获得更好性能（同步开销、上下文切换开销）

2. 进程上下文切换步骤

1. **陷入内核**：必须从用户态进入内核态
2. **保存上下文**：保存当前进程的寄存器上下文（PC、PSW、通用寄存器等）到PCB
3. **切换地址空间**：更新页表基址寄存器，TLB失效
4. **选择新进程**：调度算法选择下一个要运行的进程
5. **恢复上下文**：恢复新进程的寄存器上下文
6. **返回用户态**：跳转到新进程的程序计数器

重要：进程上下文切换**一定会**陷入内核，但陷入内核**不一定会**导致进程切换（例如系统调用返回原进程）

3. 用户级线程 vs 内核级线程

特性	用户级线程	内核级线程
线程管理	用户空间的线程库	操作系统内核
内核感知	内核不知道线程存在	内核完全感知
阻塞行为	一个线程阻塞，整个进程阻塞	一个线程阻塞，其他线程可继续运行
多处理器	不能同时运行在多个CPU上	可以并行运行在多个CPU上
切换开销	非常小，无需陷入内核	较大，需要进程上下文切换

多对多模型：多个用户级线程映射到多个内核级线程 → 既保留灵活性，又避免单线程阻塞问题

4. 自旋锁（Spinlock）

- **工作原理**：线程不进入阻塞，而是循环检查锁是否可用
- **适用场景**：临界区执行时间**很短**的场景
- **不适用场景**：临界区执行时间长 → 浪费CPU
- **优点**：避免上下文切换开销
- **缺点**：忙等待浪费CPU，可能导致优先级反转

5. PV操作（信号量）

- **P操作 (wait)**: 申请资源, $s.count--$, 如果 $s.count < 0$ 则加入等待队列 $s.queue$ 并阻塞
- **V操作 (signal)**: 释放资源, $s.count++$, 如果 $s.count \leq 0$ 则从 $s.queue$ 唤醒一个等待进程
- **原子性**: PV操作必须是原子操作 (不可中断)
- **注意**: P操作顺序会影响信号量最终值吗? **不会!** $S = S - 1 - 1$, 结果与顺序无关

6. Peterson算法 (经典互斥算法)

```
void enter_region(int process) {
    int other = 1 - process;
    interested[process] = TRUE;    // 表示我想进入
    turn = process;                // 主动让优先级给对方
    while (turn == process && interested[other] == TRUE); // 等待对方不需要或谦让给我
}

void leave_region(int process) {
    interested[process] = FALSE;   // 表示我已离开临界区
}
```

7. Dekker算法 (最早的正确互斥算法)

核心思想: 设置标志位表示想进入, 同时设置turn变量表示"轮到谁", 通过主动谦让避免死锁

8. 经典同步问题: 读者写者 (变种)

2025年考题: 公园售票系统

- **查询者Q**: 读操作, 允许多个Q并发读
- **购票者B**: 写操作, 只能互斥访问
- **额外约束**: 按到达顺序访问, 余票为0时不允许写
- **解法要点**: 使用互斥信号量保护写, 使用计数器控制读者并发, 保证读写公平

四、超高频必考点 (出现 ≥ 3 次)

1. 多级页表计算 ★★★★★

- 两年都考了相同类型的计算题: 1级页表 vs 多级页表的页表项数
- 记住公式: 32位4KB页 $\rightarrow$ 页内偏移12位 $\rightarrow$ 1级页表需要 $2^{20} = 1M$ 个PTE

2. 页面置换算法计算 ★★★★★

- 两年都考了FIFO缺页次数计算
- 还考了CLOCK/LRU的对比

3. PV操作/信号量 ★★★★★

- 选择、判断、大题全覆盖
- 理解P/V对count和queue的影响

- 能用test-and-set实现自旋锁保护的PV操作

4. 进程上下文切换 ★★★★★

- 选择、判断、简答全覆盖
- 关键点："一定会陷入内核" vs "陷入内核不一定会切换"

5. 段页式地址转换综合题 ★★★★★

- 两年几乎相同的大题类型
- 需要掌握：段表查询 → 页表查询 → 权限检查 → 物理地址计算
- 注意：有效位、只读位检查，Invalid段直接Error

五、复习优先级建议

第一优先级（必考）

1. 多级页表计算（1级 vs N级的PTE数量）
2. 页面置算法手工模拟计算（FIFO/LRU/CLOCK）
3. 段页式地址转换流程（综合题7分）
4. 页表自映射地址计算（5分大题）
5. PV操作实现同步互斥（10分大题）
6. 进程与线程的区别联系（简答题）
7. 进程上下文切换步骤（简答题）

第二优先级（高频）

1. 覆盖与交换的区别
2. 抖动的原因与解决
3. 各种页面置算法的特点对比
4. 信号量机制的理解
5. Peterson/Dekker算法填空
6. 可变分区分配算法
7. 可执行文件内存分段（DATA/BSS/STACK）

第三优先级（理解为主）

1. 操作系统架构（微内核vs宏内核）
2. 批处理系统特点
3. 反置页表优点
4. 快表工作原理
5. 局部性原理
6. 用户级线程vs内核级线程

六、易错点提醒

1. 覆盖是在同一程序内部进行的，不是不同作业之间
2. 增加可运行线程数量不能解决抖动，反而可能加剧

3. 多级页表级数越多，访存性能越差（因为需要多次查页表）
 4. **fork**后父子进程同一变量虚拟地址相同，但物理地址不同（写时复制前可能共享物理页）
 5. 线程虽然共享地址空间，但必须有自己独立的栈
 6. 用户级线程不能修改自己的页表项（页表是内核管理的）
 7. 两个不同进程的页表项可能内容相同（共享内存时）
 8. 虚拟地址空间不一定大于物理地址空间（取决于具体实现）
-

文档生成时间：2026年4月27日